

Ôn tập cuối kì

1. Basic Concepts
2. Flow of Control
3. FUNCTIONS
4. Arrays
5. Character Arrays (String)
6. Struct(ures)
7. Files
8. Pointer
9. Linked list
10. Stack and Queue

1. Basic Concepts

- Variables(biến): Là tên để có thể gán nhãn cho dữ liệu trong bộ nhớ, dùng để lưu trữ giá trị
 - Các quy tắc để đặt tên biến: Chữ hoa và chữ thường là khác nhau(ví dụ hi khác với Hi). Không được trùng với những từ đã được dùng trong máy tính(ví dụ if, for). Chỉ được gồm kí tự là chữ cái, dấu "_" và số, tuy nhiên số không được ở đầu.
 - Constant Variables: biến hằng, dùng để lưu trữ những giá trị không đổi.
- Data types:
 - Simple, built-in data types (e.g., integer, double) (được build sẵn chỉ cần lấy ra dùng)
 - User-defined (e.g., struct, class) (tự phải build lại)
 - Others (e.g., pointer in C++)

```
/*
```

```
Boolean: Chỉ có 2 giá trị là true hoặc false (1 hoặc 0)
```

```
Trong C++ được kí hiệu là bool
```

```
*/
```

0(false), -1(true), 5(true)

/*

Character: Tượng trưng cho những kí tự trong máy tính (có thể xem bảng ascii để biết thêm) có 256 kí tự

Tên trong C++ là char

*/

/*

Integer: Số nguyên có 2 loại là unsigned(không dấu) và signed(có dấu)
một số giới hạn như sau:

int: $-2^{31} \rightarrow 2^{31} - 1$

unsigned int: $0 \rightarrow 2^{32} - 1$

short int: $-2^{15} \rightarrow 2^{15} - 1$

.....

*/

/*

Real Numbers: số thực gồm 2 loại chính là float(tốn 4 byte dữ liệu để lưu) và double(8 byte)

*/

/*

Constant Variables: syntax: const DataType VariableName

*/

const double PI = 3.14;

/*

Arithmetic Operators: +, -, *, /, %(modulus)

*/

/*

Unary Operators: chỉ có 1 toán hạng ví như ++ và --

*/

++x; --x; // tăng(giảm) giá trị của x trước

x++; x--; // tăng(giảm) giá trị của x sau

/*

Bitwise Operators: Những thao tác liên quan đến nhị phân. Hiểu là chúng ta sẽ chuyển số về dạng nhị phân rồi thực hiện.

and(&), or(|), xor(^), not(~), shift left(<<), shift_right(>>)

-Đối với những phép and or xor not thì chúng ta sẽ thực hiện trên từng bit một

-Với phép and thì cả 2 bit đồng thời bằng 1 thì kết quả ra 1 còn lại ra 0

-Với phép or thì chỉ cần 1 trong 2 bit bằng 1 thì kết quả ra 1

-Với phép xor thì duy nhất 1 bằng 1 thì kết quả ra 1 còn lại ra 0

-Có 1 điều thú vị với phép xor là $a^a = 0$ và $a^0 = a$

-Phép not thì đơn giản là đổi bit lại 0 thành 1, 1 thành 0

-shift left 1 lần tương đương với việc bạn thêm 1 số 0 vào phía bên phải của số hiện tại ở dạng nhị phân

tương đương với việc nhân 2

-shift right 1 lần thì bạn bỏ đi bit ở ngoài cùng bên phải.

*/

/*

Relational Operators: Kiểm tra mối liên hệ giữa 2 toán tử(operand)

Trong C++: ==, !=, >, <, >=, <=

*/

```

/*
Logical Operators: dùng để kiểm tra một biểu thức là đúng hay sai(trả về true
hoặc false)

                                Trong C++: &&, ||, !

*/

/*
Variable Declaration: Syntax: DataType Name;
*/
// Nhớ lưu ý cách đặt tên biến đã nói ở trên nhé
int a;
double b, c, d;
//Để có thể sử dụng 1 biến thì phải khai báo trước

/*
Assignments operators: phép gán

                                Variable = expression;

*/
int x, y;
x = 5;
y = x + 5;

```

2. Flow of Control

- If else:

```

if (<Condition>)
    <Statement>;
else if(<Condition2>)
    <Statement2>;
.....
else {
    <Statements>;
}

```

Đây là cấu trúc rẽ nhánh. Có thể lấy ví dụ với một bài cơ bản sau: Cho bạn một số điểm bạn cần xác định điểm đó là loại gì?

else if sẽ được dùng nếu bạn có ≥ 3 trường hợp.

Chúng ta sẽ cố gắng chia TH ra:

- + $diem < 5$: rot mon
- + $5 \leq diem < 6.5$: trung bình
- + $6.5 \leq diem < 8$: khá
- + $8 \leq diem$: giỏi

Rồi dựa vào đó chúng ta sẽ có đoạn code dưới đây:

```
if(diem < 5) cout << "rot mon";  
else if(diem < 6.5) cout << "trung binh";
```

Bởi vì nếu check được điều kiện này có nghĩa điều kiện ở trên không đúng

⇒ $diem \geq 5$

⇒ chỉ cần check 1 TH

```
else if(diem < 8) cout << "kha";  
else cout << "gioi";
```

- **NOTE:** Lưu ý rằng các lệnh if có thể được lồng nhau và lệnh else sẽ được ghép với lệnh if gần nhất. Bạn thử đoán thử kết quả có đoạn code dưới đây ra bao nhiêu?

```
int a = 5;  
int b = 10;  
  
if (a != 5)  
  
    if(b == 10) cout << "b is double of a";  
  
else cout << "nothing"
```

Nó sẽ không in ra gì cả do else sẽ match với lệnh if thứ 2

Đó là lí do tại sao bạn nên code nó thẳng hàng với nhau để tránh nhầm lẫn hoặc bỏ vào trong {} để đảm bảo an toàn.

Cố gắng lưu ý giữa = và == nhé, 1 cái là phép gán 1 cái là Relational Operators

- Ternary Operator (?): Nó sẽ chứa 3 toán hạng với cấu trúc như sau:
 - <Condition> ? <Expression1> : <Expression2>;
 - nếu điều kiện đúng thì trả về biểu thức 1, ngược lại trả về biểu thức 2

Ví dụ check xem số a có >= 10 hay không thì có thể dùng:

```
(a >= 10) ? "True" : "False";
```

nếu a >= 10 thì sẽ trả về True
nếu không thì sẽ trả về False

- Switch-case
 - Hiểu đơn giản là nó sẽ đưa ra 1 biểu thức rồi mỗi case của mình là 1 TH, nếu là nó match với kết quả của biểu thức thì đoạn code trong đó được thực thi.

```
switch (<Variable/Expression>) {  
  
    case <Value1>:<Statement1>; break;  
    case <Value2>:<Statement2>; break;  
    ...  
    default: <StatementN>;  
  
}
```

- **NOTE:** Có 1 điều siêu lưu ý ở đây chính là chữ break, nếu không có chữ break, thì 1 khi đã match được với 1 case nào đó thì nó sẽ chạy luôn những

case dưới mà không cần phải check xem có match hay không nữa. ⇒ phải lưu ý đến chỗ break. Nó sẽ chạy đến khi nào gặp chữ break hoặc cái cuối cùng.

Kiểm tra số nhập vào là chẵn hay lẻ.

```
int a; cin >> a;
```

```
switch(a % 2) {  
    case 0:          // Các giá trị có thể của biểu thức  
        cout << "so chan";  
        break;       // Lưu ý kỹ chỗ này, nếu không có break thì sẽ có  
chuyện xảy ra  
    case 1:  
        cout << "so le";  
        break;  
  
    default:         // nên có phần này để tránh rơi vào những trường hợp  
p mình liệt kê thiếu  
                                // giá trị có thể của biểu thức trên thì nó  
đều sẽ rơi vào default hết.  
        break;  
}
```

Nếu như không có chữ break mà số a = 10 thì điều thú vị chương trình sẽ cout ra: sochansole (lí do ở NOTE trên)

- Loop structure:
 - while statement: Luôn check điều kiện trước khi chạy, và lưu ý rằng nếu đặt điều kiện không chuẩn có thể dẫn đến chạy vô hạn

```
while (<LoopCondition>){  
  
    <Statement1>;  
    <Statement2>;  
}
```

```
...  
}
```

Ví dụ in ra các số từ 1 đến n;

```
int n; cin >> n;  
int cur = 1;  
// n = 0  $\Rightarrow$  1 <= 0  
while(cur <= n) {  
    cout << cur << " ";  
    cur++;  
}
```

Tuy nhiên nếu $n = 0$ hay số bé hơn 0 thì đều sẽ không in ra gì do điều kiện luôn được check trước mỗi khi chạy.

- do ... while statement: Đối với cái này thì những lệnh trong do sẽ được thực hiện 1 lần. Sau đó rồi mới bắt đầu check điều kiện rồi y chang while chỉ khác ở chỗ là chắc chắn được thực hiện ít nhất 1 lần.

```
do{  
  
    <Statement1>;  
    <Statement2>;  
  
    ...  
}  
while(<LoopCondition>;
```

Trường hợp áp dụng khi mà mình muốn vòng lặp đó ít nhất phải được thực hiện 1 lần bất kể điều kiện gì.

```
int number;
```

```
// Vòng lặp do-while đảm bảo ít nhất một lần thực thi
do {
    cout << "Enter a number between 1 and 10: ";
    cin >> number;

    if (number < 1 || number > 10) {
        cout << "Invalid input. Please try again.\n";
    }
} while (number < 1 || number > 10); // Điều kiện kiểm tra

// Khi nhập hợp lệ
cout << "You entered a valid number: " << number << endl;
```

- for statement: Đầu tiên sẽ là khởi tạo giá trị để bắt đầu, sau đó là điều kiện rồi đến bước nhảy (nghĩa là biến khởi tạo sẽ tăng bao nhiêu sau mỗi lần lặp. Nó sẽ thực theo quy trình: khởi tạo → kiểm tra điều kiện → thực hiện phần bên trong → thay đổi biến → kiểm tra điều kiện →

```
for (<Init>;<LoopCondition>;<Steps>){

    <Statement1>;
    <Statement2>;

    ...

}
```

Ví dụ cout các số từ 1 → n. Đầu tiên khởi tạo số đầu tiên bằng 1. điều kiện là số phải ≤ n, bước nhảy: do muốn cout các số liên tiếp nên bước nhảy sẽ là 1.

```
int n; cin >> n;

for(int i = 1; i <= n; i++) cout << i << " ";
```

// Có 2 lệnh liên quan đến phần này là break và continue, break thì sẽ khiến mình thoát khỏi vòng for luôn,
// còn continue thì chỉ qua qua lần lặp đó và tiến tới lần lặp tiếp theo.

bây giờ mình chỉ muốn cout các số lẻ thì sao??? Mình sẽ bỏ qua những số chẵn dùng lệnh continue

```
for(int i = 1; i <= n; i++) {  
    if(i % 2 == 0) continue;  
    cout << i << " ";  
}
```

1 3 5 7 9

```
for(int i = 1; i <= n; i += 2) {  
    cout << i << " ";  
}
```

Bây giờ chỉ muốn cout đến một max nào đó??? Nếu đạt ngưỡng thì sẽ break không lặp nữa dùng lệnh break

```
for(int i = 1; i <= n; i++) {  
    if(i > maxx) break;  
    cout << i << " ";  
}
```

Những ví dụ trên chỉ mang tính chất minh họa và hoàn toàn có cách làm dễ hơn.

3. FUNCTIONS

- Functions: Giờ thử tưởng tượng rằng bạn cần tính $5! + 6! + 7!$ bạn có thể thực hiện 3 vòng for để có thể tính toán (hoàn toàn có khả năng). Nhưng mà nếu như bạn muốn tính $1! + 2! + \dots + 100!$ thì sao? $\Rightarrow$ code sẽ rất dài.

- Ở đây function sẽ cho bạn tính chất sử dụng lại (reusable) nghĩa là bạn chỉ cần làm 1 lần và sau đó sử dụng lại
- Hàm (function) là những đoạn code mà chỉ được chạy khi được gọi.

```
<DataType> FunctionName(<ArgumentList>) {
```

```
    // body of function
    return <value>;
}
```

Nếu như mà không return về giá trị gì hết thì Datatype của nó sẽ là void, nếu như là void thì sẽ có lệnh return, cũng giống ý nghĩa của break trong for sẽ giúp bạn thoát khỏi hàm, không thực hiện những phần còn lại của hàm

```
void ghi_ra() {
    cout << "Hello";
    return; // dùng để thoát khỏi hàm, do datatype là void nên không trả về gì hết
}
```

Nếu như mà đổi return lên trên thì sao???

```
void ghi_ra() {
    return; // Hàm đã thoát ra ở đây rồi nên không có gì được cout
    cout << "Hello";
}
```

Còn Trường hợp datatype không phải void??

```
int factorial(int n) {
    int ans = 1;
    for(int i = 1; i <= n; i++) ans = ans * i;
    return ans;
}
```

Nếu như mà không có return thì sao??? Trong 1 số TH may mắn thì bạn sẽ chạy bình thường, nhưng mà nó được xem là Undefined Behavior có thể khiến chương trình n hận giá trị rác, thậm chí là crash

```
int factorial(int n) {  
    int ans = 1;  
    for(int i = 1; i <= n; i++) ans = ans * i;  
} // lỗi
```

Bắt buộc rằng giá trị trả về phải là kiểu datatype

```
int factorial(int n) {  
    char tmp = 'a';  
    return tmp; // lỗi  
}
```

- Khi code thì chúng ta có thể chia hàm ra làm 2 phần: 1 là declaration (kiểu là bạn khai báo nó nhưng mà chưa cài đặt gì bên trong), 2 definition, đây chính là phần mà bạn sẽ cài đặt hàm của bạn(viết code)

```
int factorial (int n); // khai báo kiểu để chương trình của bạn biết là có hàm này
```

```
int main() {  
}
```

```
int factorial(int n) {  
    int ans = 1;  
    for(int i = 1; i <= n; i++) ans = ans * i;  
    return ans;  
}
```

Lí do tại sao lại cần như vậy?? Tại vì nếu như chúng ta có 3 hàm A, B, C. tr

ong code của A cần hàm B, trong code hàm B cần hàm C, trong code hàm C cần hàm A. Và để có thể sử dụng thì hàm đó phải được khai báo trước. Vậy chúng ta sẽ nghĩ ra thứ tự sắp xếp A, B, C như thế nào để đảm bảo cả 3 hàm đều chạy??? Không tồn tại do nó là vòng tròn không thể tìm ra thẳng bắt đầu.

⇒ Mình nên chia thành 2 phần như trên.

- Scope: Bạn có thể hiểu đơn giản là phần nằm giữa 2 dấu { } là được. Dựa vào đó thì có thể biến của mình thành 2 loại là Global và Local
 - Global: được đặt ở ngoài hàm, kể cả hàm main có thể được dùng ở mọi nơi
 - Local: được đặt ở trong 1 hàm hoặc { } thì nó chỉ có tác dụng trong vùng đó thôi.

```
#include <iostream>
using namespace std;
```

```
int x;
```

```
int add(int a) {
    int tmp;
    return a + x; // được sử dụng do x là biến toàn cục.
}
```

```
int add_1(int a) {
    return a + tmp; // tmp này đâu ra???? do tmp ở hàm add chỉ có tác dụng ở trong hàm đó thôi
}
```

hãy để ý ngoặc nhọn để có thể thấy được vùng nó được sử dụng (scope)

- Parameter passing: Giờ sẽ bàn đến vấn đề tham số mình truyền vào. Sẽ có 2 loại truyền vào, 1 là call by value, 2 là call by reference.
 - call by value: Có nghĩa là bạn truyền vào 1 "thứ", chương trình sẽ clone ra thứ y chang thứ đó và bạn có thể thao tác tùy ý với cái mới clone ra mà không ảnh hưởng đến thứ bạn truyền vào. Được dùng khi bạn không muốn thay đổi giá trị truyền vào.

```
void add(int a) {  
    a += 5;  
}  
  
int main() {  
    int a = 10;  
    add(a);  
    //a = 10 do mình truyền a vào nhưng mà nó chỉ clone ra a mới và không thay đổi giá trị thật của cái mình truyền vào  
}
```

- Call-by-Reference: Lúc này bạn pass vào địa chỉ(address) của tham số truyền vào. Thường sẽ thêm & trước tham số. Điều này sẽ dẫn đến giá trị thật của tham số (lí do sẽ được làm rõ khi mình học đến con trỏ).

```
void add(int &a) {  
    a += 5;  
}  
  
int main() {  
    int a = 10;  
    add(a);  
    //a = 15 do mình truyền a theo phương thức call by reference  
}
```

Cách này thường dùng để tiết kiệm bộ nhớ cũng như thời gian chạy do bạn không cần clone ra cái gì cả mà

làm trực tiếp trên tham số truyền vào.

Có 1 trick cho cái này là nếu bạn không muốn thay đổi giá trị mà muốn chạy nhanh thì bạn có thể sử dụng const để chắc chắn rằng trong đoạn code của bạn không có bất kì hành vi nào làm thay đổi tham số bạn truyền vào

```
void add(const int &a)
```

đây là Call-by-Reference và sẽ đưa ra cảnh báo nếu có bất kì đoạn code nào khiến a thay đổi.

!!! Nó sẽ đưa ra cảnh báo nhé chứ không phải là bạn muốn thay đổi a như nào cũng được

```
void add(const int &a) {  
    a = a + 5; // sai cho a đang là const  
}
```

- Recursive Functions: Hàm tự gọi lại chính nó. Bắt buộc phải có 1 điểm dừng hay gọi là base do nếu không có thì nó cứ gọi chính nó vô hạn lần.

```
int factorial(int n) //return n!  
{  
    if(n <= 1) return 1; // base  
    //n! = (n - 1)! * n  
  
    return n * factorial(n - 1);  
}
```

- Function Overloading: Nghĩa là nếu 2 hàm mà có cùng tên nhưng mà khác các tham số nhập vào thì vẫn tính là 2 hàm, tuy nhiên nó chỉ quan tâm đến tên và tham số thôi nhé còn loại hàm trả về thì không quan tâm.

```
int test();  
void test(int);  
void test(float);  
int test(float); // invalid do đã giống có test(float) ở trên rồi (không quan tâm giá trị trả về của hàm)  
float test(); // invalid tương tự.  
int test(int,int);
```

Default Arguments: là những tham số mặc định. Khi gọi hàm thì bạn có thể bỏ qua những tham số đó hoặc ghi đè lên cũng được.
Nghĩa là nếu bạn không truyền giá trị thì nó lấy giá trị mặc định.

Lưu ý: Nếu tham số được khai báo là mặc định thì tất cả phía sau cũng đều phải là mặc định hết

```
void func1(int x, double z = 3.14, char y); //wrong
```

```
//Function as an Argument
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
void add_3(int Func(int, int), int a, int b, int c) {  
    return Func(a, b) + c;  
}
```

```
//lúc gọi thì chỉ cần bỏ tên vào  
cout << add_3(add, a, b, c)
```

4. Arrays

- One-dimensional Arrays: mảng một chiều. Vấn đề là giờ bạn muốn lưu 100 số không lẽ bạn lại khai báo 100 biến??? Thì array sẽ giúp bạn. Đây là một cấu trúc mới giúp lưu trữ một chuỗi các dữ liệu và có thể truy cập nhanh.
- Giả sử như bây giờ bạn cần 1 ô vuông để lưu trữ 1 đồ vật (biến) thì array sẽ như 1 dãy ô vuông liên tiếp vậy giúp bạn lưu trữ và có thể truy cập nhanh (do nó nằm liên tiếp nên bạn có thể gọi trực tiếp ví dụ bạn muốn đến hộp 4 thì gọi 4 là được). Thì điểm chung rằng những phần tử này phải có cùng 1 kiểu dữ liệu (ví dụ array of int, array of char,..). Minh truy cập nhanh dựa vào chỉ số của nó. kiểu dữ liệu thứ 10 thì ô 10 (nếu đánh số từ 1).
- Tuy nhiên mảng sẽ thường được đánh số từ 0, ví dụ khai báo a[5] thì chỉ được sử dụng a[0], a[1], a[2], a[3], a[4] (đúng 5 phần tử)

```
<DataType> Name[<SIZE>;
```

```
<DataType> Name[<SIZE1>][<SIZE2>]...[<SIZEN>;
```

```
// Truy cập vào phần tử ở trong mảng
```

```
Name[<Index1>][<Index2>]...[<IndexN>]
```

```
// Có 1 điều lưu ý rằng 0 <= index_1 < Size_1, tương tự với những index khác
```

```
// Không được sử dụng array_1 = array_2
```

```
// Không sử dụng cin >> array_1
```

```
// Không sử dụng cout << array_2
```

```
// Lí do sẽ được hiểu rõ hơn khi học pointer
```

```
int a[4] = {1, 2, 3, 4};
```

```
int b[4];
```

```
cin >> a //wrong
```

```
cout << a // wrong
```

```
b = a //wrong
```

```
// should do
```

```
for(int i = 0; i < 4; i++) b[i] = a[i];  
for(int i = 0; i < 4; i++) cin >> a[i];  
for(int i = 0; i < 4; i++) cout << a[i];
```

- 1D Arrays as Function Parameters: Mảng chỉ được truyền vào thông qua địa chỉ $\Rightarrow$ hoàn toàn có thể thay đổi giá trị của array ở trong function. Và size của mảng sẽ được bỏ qua, nếu bạn cung cấp thì chương trình cũng sẽ bỏ qua. Không dùng & nhé.

```
void zeroFill(int arr[], int n) {  
    for(int i = 0; i < n; i++) arr[i] = 0;  
}
```

```
void zeroFill(int *arr, int n) {  
    for(int i = 0; i < n; i++) arr[i] = 0;  
}
```

```
// sử dụng const nếu bạn không muốn nó thay đổi
```

```
void zeroFill(const int *arr, int n) {  
    for(int i = 0; i < n; i++) arr[i] = 0; // sẽ thông báo lỗi cho bạn  
}
```

```
// Còn kích thước của mảng 1 chiều thì sao?? Giờ bảng sẽ lưu N phần tử thì mỗi  
i phần tử có size thì sẽ là N * size  
// bạn có thể dùng lệnh sizeof(<tên>) để biết được kích thước.
```

```
int a[10];  
cout << sizeof(a); // 40 do mỗi int sẽ là 4 byte
```

- Two-dimensional arrays: Tưởng tượng nó giống như một hình chữ nhật thì một ô sẽ được đặc trưng bởi 2 chỉ số là tọa độ hàng và cột (hàng trước cột sau)

```
<DataType> Name[<SIZE1>][<SIZE2>]
```

// Khi mà truyền mảng 2d vào trong function thì kích thước cột phải được xác định còn hàng sẽ được bỏ trống.

```
void zeroFill(int a[][10], int nRow, int nCol) {
    ...
}
```

// Còn kích thước của mảng 2 chiều thì sao?? Giờ bảng sẽ lưu $N * M$ phần tử thì mỗi phần tử có size thì sẽ là $N * M * \text{size}$
 // bạn có thể dùng lệnh `sizeof(<tên>)` để biết được kích thước.

```
int a[10][10];
```

```
cout << sizeof(a); // 400 do mỗi int là 4
```

- Thử sức với một bài bài tập:

Câu 1 (2 điểm): Cho mảng A nguyên dương có chiều dài N , viết hàm có nguyên mẫu như sau `int reduceArray(int A[], int& N)` để rút ngắn A bằng cách sát nhập các số nguyên tố liên tiếp nhau trong A lại thành tổng của chúng, đồng thời trả ra số lượng chuỗi con số nguyên tố liên tiếp.

Ví dụ: $A = \{ \underline{5} \underline{2} \underline{6} \underline{3} \underline{7} \underline{11} \underline{20} \}$, $N = 7 \Rightarrow A = \{ \underline{7} \underline{6} \underline{21} \underline{20} \}$, $N = 4$ và hàm trả ra 2

$A = \{ \underline{5} \underline{2} \underline{6} \underline{7} \underline{11} \underline{8} \underline{13} \underline{5} \underline{3} \underline{15} \}$, $N = 10 \Rightarrow A = \{ \underline{7} \underline{6} \underline{18} \underline{8} \underline{21} \underline{15} \}$, $N = 6$ và hàm trả ra 3

- Điều đầu tiên mình cần phải nhận xét được ở bài này là số lượng phần tử của mảng sau cùng luôn $\leq$ số lượng phần tử ban đầu. Do đó chúng ta có thể dùng chính mảng a để lưu mà không cần thêm gì nữa. (1)
- Nếu check 1 số nguyên tố thì dễ, nhưng mà nhiều số thì chúng ta nên nghĩ đến sử dụng function để đoạn code ngắn và không rối $\Rightarrow$ sẽ tạo một function để check snt

- o Từ (1) hãy đưa ra nhận xét được rằng, ví dụ mình xét đến vị trí 5 nghĩa là các số vị trí từ 1 → 5 mình không còn quan tâm nữa do đã biết giá trị rồi ⇒ sẽ tận dụng nó để lưu đáp án. Và cũng từ đó xét 5 số đầu tiên thì cũng chỉ có tối đa được 5 số xuất hiện trong mảng đáp án ⇒ dùng 1 biến để lưu vị trí lưu đáp án hiện tại.

```
bool check_snt(int n) {
    if(n < 2) return 0;
    if(n == 2) return 1;
    for(int i = 2; i * i <= n; i++) if(n % i == 0) return 0;
    return 1;
}

int reduceArray(int A[], int& n) {
    int cur = 0; // vị trí lưu đáp án
    int ans = 0; // lưu số lượng dãy con nguyên tố
    for(int i = 0; i < n; i++) {
        if(check_snt(a[i]) {
            int pos = i;
            int sum = a[i];
            while(pos + 1 <= n && check_snt(a[pos + 1])) { // dùng để tìm
m dãy con nguyên tố liên tiếp
                pos++;
                sum += a[pos];
            }

            a[cur] = sum;
            cur++;
            ans++;

            i = pos; //lưu ý rằng phải gán như thế này do các số từ i → pos
os đều đã được xử lí rồi
        }
        else {
            a[cur] = a[i];

```

```

        cur++;
    }
}
n = cur;
return ans;
}

```

- Một bài tập liên quan đến mảng 2 chiều: đánh số cho các ô theo chiều xoắn ốc từ ô (0, 0), từ ngoài vào trong theo chiều kim đồng hồ.

Bạn sẽ cần nhận xét cho bài này như sau: Thật ra bạn hoàn toàn có thể xem những quá trình di chuyển như vậy thành những hình tròn cho dễ hình dung. Thì lần đầu sẽ là đi ở vòng tròn bên ngoài cùng, sau khi đi hết thì đến vòng thứ 2,... Vậy có phải bạn chỉ cần tìm ra điểm bắt đầu cho mỗi vòng tròn như vậy là bài này sẽ giải quyết dễ hơn.

nếu bạn vẽ ra một lúc thì sẽ thấy rằng đầu tiên sẽ ở ô (0, 0) sau đó là (1, 1), (2, 2),....

Sau khi xác định được ô bắt đầu thì bạn sẽ tiến thành đi sang phải đến khi gặp biên rồi xuống dưới đến khi gặp biên rồi sang trái đến khi gặp biên, rồi lên trên đến khi gặp biên.

Lưu ý rằng sau mỗi vòng thì biên trên dưới trái phải đều vào 1 bậc nghĩa là trái + 1, phải - 1, trên + 1, dưới - 1

```

int top = 0, bottom = rows - 1;
int left = 0, right = cols - 1;
int counter = 1;

while (top <= bottom && left <= right) {
    // Đi từ trái sang phải

```

```

for (int i = left; i <= right; i++) {
    matrix[top][i] = counter++;
}
top++; // Do hàng top này đã được đánh số hết rồi nên là top++

// Đi từ trên xuống dưới
for (int i = top; i <= bottom; i++) {
    matrix[i][right] = counter++;
}
right--; // Do hàng cột right này đã được đánh số hết rồi nên là right--

// Đi từ phải sang trái
if (top <= bottom) {
    // Ở đây phải if do mình chỉ check điều kiện top <= bottom ở trên w
    hile còn ở phía trên
    // thấy rằng top đã tăng lên làm nguy cơ điều kiện này không còn đ
    úng
    for (int i = right; i >= left; i--) {
        matrix[bottom][i] = counter++;
    }
    bottom--;
}

// Đi từ dưới lên trên
if (left <= right) {
    for (int i = bottom; i >= top; i--) {
        matrix[i][left] = counter++;
    }
    left++;
}
}

```

5. Character Arrays (String)

```
char StudentID[9]; // "24127999\0"
```

//sẽ có 1 kí tự cuối cùng là null-terminated ('\0') để biết rằng xâu kết thúc tại đó

'a' được hiểu là char

"a" được hiểu là 2 kí tự 'a' và '\0'

```
s[10] = "2312"
```

Thì sizeof(s) = 10

s[0] = '2', s[1] = '3', s[2] = '1', s[3] = '2', s[4] = '\0', còn lại trống

- input sử dụng cin
 - Nếu như bạn muốn nhập vào 1 xâu mà không có dấu cách thì đơn giản bạn hoàn toàn có thể dùng cin >> s
 - Còn nếu muốn nhập luôn cả dấu cách thì là cin.getline(s, size)

Lí do tại sao với những loại mảng int thì không thể dùng cin >> a mà mảng char thì lại được??

Đơn giản do char được C++ hỗ trợ

```
char s[10];
```

```
cin >> s; // nếu gặp dấu cách sẽ dừng
```

Ví dụ xâu "abd nhh" thì s chỉ nhận "abd"

Nếu như mà muốn nhập hết chứa luôn dấu cách thì dùng

```
cin.getline(s, 10) // s = "abd nhh"
```

Lệnh `cin.getline` hoạt động như thế nào???

Lệnh này giúp bạn đọc xâu đến khi nào gặp kí tự xuống dòng `'\n'` hoặc đủ số lượng kí tự

`cin.getline(s, size)` // lưu ý là size là đã tính luôn cả kí tự null rồi nhé

Nghĩa là nếu bạn muốn lưu 100 kí tự thì size nên là 101

Sẽ có điều lưu ý cho mấy bạn khi code:

+ Bạn thấy rằng nếu `cin.getline()` gặp kí tự `'\n'` thì sẽ bị dừng. Nếu chúng ta `cin` 1 giá trị nào đó trước khi `cin.getline()` thì khả năng rất cao là kí tự `'\n'` vẫn chưa được đọc ra khỏi luồng `cin` và nó sẽ bị `cin.getline()` đọc $\Rightarrow$ mất dữ liệu

```
int a;  
char s[10];
```

```
cin >> a; // còn dữ '\n';
```

```
cin.getline(s, 10); // đọc '\n' và dừng ngay  $\Rightarrow$  s trống  $\Rightarrow$  mất dữ liệu
```

Cách khắc phục là gì??? bỏ đi kí tự `'\n'` bằng cách `cin.ignore()`;

```
cin >> a;  
cin.ignore();  
cin.getline(s, 10);
```

Tuy nhiên nếu trước đó là `getline` thì không cần do đã đọc luôn `'\n'` rồi.

```
char a[10], b[10];
```

```
cin.getline(a);  
cin.getline(b); // không lỗi do ở a đã đọc luôn '\n' rồi.
```

- Output sử dụng cout
 - Đơn giản chỉ là cout << s là được chứ không cần cout từng kí tự.
- The C-String library
 - strcpy(s1,s2): copy từ s2 qua s1 vì thế nên size của s1 ít nhất phải bằng s2

Lưu ý nếu muốn sử dụng những hàm này phải có thư viện cstring
`#include <cstring>`

```
char s[10];
cin.getline(s, 10);
```

```
char a[10];
strcpy(a, s);
```

Bây giờ a đã giống s

- strcmp(s1, s2): trả về < 0 nếu s1 < s2, 0 nếu s1 = s2, > 0 nếu s1 > s2. Việc so sánh này sẽ dựa vào thứ tự từ điển. Nghĩa là bạn sẽ so sánh từ vị trí 0, nếu kí tự ở 2 xâu đều bằng nhau thì xét đến vị trí tiếp theo, nếu khác nhau thì xâu nào có kí tự bé hơn thì bé hơn. Nếu đã xét hết một trong hai xâu mà vẫn chưa tìm ra chỗ khác nhau thì xem xâu nào có độ dài ngắn hơn thì xâu đó bé hơn

```
"abs" < "abz"
```

```
"abc" < "abcd"
```

- strlen(s1): trả về độ dài của mảng, lưu ý không tính kí tự null

Lưu ý là độ dài của cái xâu đó, nghĩa là từ đầu đến khi nó gặp kí tự null, lệnh này khác sizeof nhé.

```
char a[10];
```

```
a = "abd";
```

```
strlen(a) = bao nhiêu????
```

```
strlen(a) = 3;
```

6. Struct(ures)

- Vấn đề đặt ra là với một học sinh sẽ cần lưu trữ nhiều thông tin, ví dụ như tên, tuổi, id, lớp, địa chỉ,...Vậy liệu rằng có thể gom hết các dữ liệu đó lại được không?
- Structure là một tập hợp các biến với các kiểu khác nhau đặt dưới một cái tên.

```
struct name{
```

```
    DataType1 MemberName1;
```

```
    DataType2 MemberName2;
```

```
    DataType3 MemberName3;
```

```
    ...
```

```
    DataTypeN MemberNameN;
```

```
};
```

```
struct Student {
```

```
    char ID[9];
```

```
    char name[100];
```

```
    char gender;
```

```
    int age;
```

```
    .....
```

```
}
```

- Vậy sẽ sử dụng nó như thế nào??

Đây là user-defined data types. Nên là mình có thể sử dụng y chang những loại data types khác nhau int hay char

```
Student a; // a is variable type Student
```

```
Student a[100]; // array of 100 Students
```

Những phần ở bên trong struct thì có thể được gọi là data member vậy thì chúng ta có thể truy cập nó như thế nào??

⇒ Sử dụng "dot operator"

```
a.ID;
```

```
a.name;
```

Nếu muốn xem size của structure thì dùng lệnh sizeof(<name>)

- Member function: Ngoài data member ra thì trong struct cũng có thể có những hàm của riêng nó gọi là member function. Mục đích của nó có thể là để khởi tạo các data cho kiểu dữ liệu đó (đây được gọi là constructor) hoặc hành động khác (method)
 - Constructor: Không có kiểu trả về kể cả void. Có tên trùng với tên của struct.
 - Method: Những hàm mà struct có thể làm được.

```
struct Student {  
    int ID;  
    char name[100];  
    Student() {}; // constructor  
  
    int getID() {  
        return ID;  
    }  
}
```

constructor thì mình sẽ tìm hiểu thêm về ở các môn sau.

Tuy nhiên đối với những bài tập có áp dụng struct thì thường có yêu cầu code phép gán (phép =) vậy sẽ làm nó như thế nào??

```
Student a, b;
```

```
.....
```

b = a; // Bạn hoàn toàn có thể làm như này vì ngôn ngữ c++ cho phép, mọi dữ liệu của a sẽ được copy sang b

NOTE: Tuy nhiên vẫn sẽ có trường hợp rằng dùng như vậy không chuẩn ⇒ TH đặc biệt xem ở bài phần POINTER.

- Cũng có 1 cách khác để chúng ta có thể cài đặt member function còn ghi trong struct chỉ là declaration. bằng cách sử dụng resolution operator(::)

```
int Student::get_ID() {  
    return ID;  
}
```

// này có thể ghi ngoài struct do có :: đã chỉ cho mình biết là get_ID là từ đâu ra.

- Bắt đầu với 1 bài tập để nắm rõ hơn cách sử dụng structure nhé

Cho thông tin của một đơn hàng được giao trong địa bàn TP. Hồ Chí Minh bao gồm:

- Mã đơn hàng: chuỗi ký tự (tối đa 17 ký tự).
- Tên người nhận: chuỗi ký tự (tối đa 15 ký tự).
- Địa chỉ: chuỗi ký tự (tối đa 50 ký tự).
- Số điện thoại: chuỗi ký tự (tối đa 10 ký tự).
- Khối lượng: số thực (đơn vị kilogram).

- Giờ mình thấy rằng có rất nhiều thông tin nên chúng ta sẽ cố gắng gom nó lại thành 1 nhóm → sẽ dùng struct thể hiện rằng 1 đơn hàng sẽ bao gồm các yếu

tổ sau

chuỗi kí tự thì mình sẽ sử dụng char array nhé !!

nên khai báo size + 1 vì còn kí tự null nữa.

```
struct Package {  
    char ID[18];  
    char Name[16];  
    char Address[51];  
    char PhoneNumber[11];  
    float Weight;  
  
    void read(); //file_name = input  
};
```

Rồi còn phần viết hàm của struct sẽ còn kết hợp với các kiến thức ở phía sau.

7. Files

- Làm sao để xử lí với file????
 - Có 3 thư viện để có thể đọc từ file và ghi ra file: ifstream (cho đọc từ file), ofstream (in ra từ file), fstream (cho cả 2)
 - Mở text file

```
ifstream fin_1("a.txt");
```

```
ifstream fin_1; fin_1.open("a.txt")
```

```
fstream tmp;
```

```
tmp.open("a.txt", ios::in); //lưu ý cái ios đằng sau nhé, nó sẽ đại diện cho các mode bạn sẽ sử dụng
```

```
ofstream fout_1("a.txt");
```

```
ostream fout_1; fout_1.open("a.txt")
```

Sẽ có một số mode sau:

`ios::in` mở file cho input operation

`ios::out` mở file cho output operation

`ios::binary` mở file dưới dạng nhị phân

`ios::ate` đặt vị trí đầu tiên của mình là cuối của file, nếu không set thì vị trí bắt đầu sẽ là đầu file

`ios::app` mod này cho phép bạn ghi thêm vào file đó

`ios::trunc` Nếu như tệp đó có dữ liệu trước đó thì bạn sẽ xoá dữ liệu cũ đi để mà ghi dữ liệu mới vào.

- Ghi vào file hay lấy dữ liệu từ file như thế nào??? bạn thấy rằng `cin` hay `cout` là một biến kiểu `istream`, `ostream`, thì cũng sử dụng y chang `z`, thay `cin` hay `cout` bằng tên biến bạn tạo ra thôi, ví dụ như bạn ghi là `ifstream fin`, thì thay vì `cin >> a` là nhập vào từ bàn phím thì bạn sẽ sử dụng `fin >> a`;
- Chúng ta sẽ làm một ví dụ để hiểu rõ hơn làm sao áp dụng những kiến thức trên nhé. Phạm vi kiến thức chỉ kiểm tra trong text file. Ví dụ này sẽ tiếp tục của đề bài phía trên.

- b. (10 điểm) Cho tập tin đầu vào `input.txt` chứa thông tin của các đơn hàng với:
- Dòng đầu tiên: chứa thông tin các thông số của đơn hàng
 - Mỗi dòng tiếp theo chứa thông tin của một đơn hàng cụ thể

Viết hàm đọc và lưu trữ danh sách đơn hàng của các đơn hàng. Biết rằng, cấu trúc tập tin `input.txt` như sau:

```
Ma don hang|Ten nguoi nhan|Dia chi|So dien thoai|Khoi luong
59392159392159392|Nguyen Teo|28, Le Duan, Ben Nghe, Q1|0909001001|0.3
58423158423158423|Tran Ty|34, Vo Van Tan, P5, Q3|0908123123|1.2
58522258522258522|Le Meo|220, Nguyen Xi, P26, Binh Thanh|0978123456|0.9
59283159283159283|Ly Suu|40, Bach Dang, P24, Binh Thanh|0962456567|0.5
```

Phần 0: Quy trình

Bước 1: Include thư viện `fstream`

Bước 2: Khai báo input/output stream thay vì `cin/cout` (ví dụ `ifstream fin`, `ofstream fout`)

Bước 3: Mở file (`.open(<tên file>)`)

Bước 4: Kiểm tra xem file có mở không (`.is_open()`)

Bước 5: Muốn đọc gì hay ghi gì thì dùng input/output stream bạn đã khai báo

Bước 6: Đóng file

Phần 1: đọc đầu vào từ file `input.txt`

Nhìn vào cấu trúc tệp thấy rằng chúng ta sẽ bỏ qua dòng đầu tiên.

Lệnh `getline` còn có thêm 1 tính năng

`getline(<tên mảng char>, size, delimiting_character);`

`delimiting_character`: đây là kí tự bạn gặp thì sẽ cái `getline` đó sẽ dừng và ko thêm kí tự vào `char array` nhé

⇒ Với bài trên thấy các dữ liệu được ngăn cách bởi `'|'`

⇒ Bạn hoàn toàn có thể sử dụng kí tự `'|'` để làm `delimiting_character`.

ví dụ tôi nhập vào bàn phím là "dfd|djd"
thì xâu tôi chỉ nhận đc dfd do '|' là kí tự tôi set để dừng

Hãy nhớ lại cũng có 1 điều kiện khác để dừng là '\n' hay hiểu là xuống dòng n hé.

Vậy nếu bài này sự ngăn cách không phải dấu '|' mà là dấu ',' thì sao?? thì đổi '|' thành ',' thôi.

Có 1 kiến thức nữa, là .eof(end of file). Thì nó sẽ xảy ra khi mà bạn "đọc" và thấy rằng không có gì để đọc.

Nghĩa là bạn phải đọc đã nhé rồi mới thấy không có gì để đọc rồi mới khiến <ten>.eof() = 1;

Đối với bài này sẽ thấy rằng mình không thể biết rằng số lượng đơn hàng mình cần nhập nên không thể for thuần thuần tuý được

Có thể thấy rằng nếu không còn gì để đọc thì fin.getline() có khả năng sẽ không thực hiện được ⇒ có thể dùng nó để làm điều kiện cho vòng while của mình.

Bước suy luận + phân tích đã xong đến phần code:

```
#include <fstream>
using namespace std;
```

```
int main() {
```

```
    Package package[100];
    ifstream fin;
    fin.open("input.txt");
    if(fin.is_open()) {
```

```
        char *s[1000];
        // đọc dòng đầu thấy rằng dữ liệu phần này sẽ được bỏ qua.
        file.getline(s, 1000);
```

```

int cur = 0; //vị trí để lưu gói hàng đó trong mảng
// Thì mình sẽ xử lý theo từng gói hàng)
while(fin.getline(package[cur].ID, 18, '|') {
    // Thấy rằng ID đã được nhập ở while thì bây giờ mình sẽ nhập n
hững cái còn lại.
    fin.getline(package[cur].Name, 16, '|');
    fin.getline(package[cur].Address, 51, '|');
    fin.getline(package[cur].PhoneNumber, 11, '|');
    fin >> package[cur].Weight;
    fin.ignore(); // để loại bỏ đi dấu xuống dòng ở đây tránh gây lỗi
cho những lệnh đọc sau.

    cur++; //lưu ý rằng vị trí này đã được lưu rồi nên đơn hàng mới s
ẽ được lưu ở vị trí tiếp theo
}
}
fin.close();
}

```

8. Pointer

- Giờ bạn hãy tưởng tượng bộ nhớ của bạn sẽ gồm cả tí cái ô trống. Mỗi ô trống đó sẽ gồm có 8 bit (1 byte) để có thể lưu trữ dữ liệu. Vậy thì sao làm mình biết đó là ô gì??? Như là ngôi nhà, sao bạn phân biệt đc nhà mình với nhà của bạn gái mình??? Đó chính là địa chỉ, thì mỗi ô trống sẽ có 1 địa chỉ - đó cũng chính là khái niệm cơ bản nhất của pointer.
- Pointer giúp lưu trữ địa chỉ của 1 biến nào đó. Ví dụ 1 int thì cần 4 byte, là 1 ô vuông nhỏ liên tiếp nhau thì nó sẽ lưu trữ địa chỉ của ô vuông đầu tiên ⇒ phụ thuộc vào kiểu dữ liệu của biến đó.

Syntax:

```
<datatype>* <tên>;
```

```
int* ptr;
```

```
int a = 10;  
ptr = &a; //dấu & dùng để lấy địa chỉ của a trong bộ nhớ.
```

Bây giờ nếu mình có con trỏ mà muốn cout ra giá trị tại vị trí đó thì làm kiểu gì???

Hãy dùng * trước đó, đây được hiểu là de-reference

```
cout << *ptr; // đây chính là a
```

- Pointer chỉ chấp nhận những địa chỉ của cùng loại

```
float a;
```

```
int *ptr = &a; //error
```

nullptr: Là 1 con trỏ rỗng không lưu trữ gì cả.(Empty address)

nếu như bây giờ tôi muốn khai báo 2 cái pointer thì làm như này được không?

```
int* a, b; thì nó chỉ nhận a là pointer còn b là int
```

⇒ phải khai báo như sau:

```
int *a, *b;
```

- Vậy nếu bây giờ con trỏ loại 1 cái struct thì sao????

```

1 #include <iostream>
2 using namespace std;
3 struct nhan {
4     int x, y, z;
5 } a;
6 int main() {
7     nhan* p = &a;
8     //type 1:
9     cout << (*p).x;
10
11     //type 2:
12     cout << p -> x;
13 }

```

- Passing pointer to function:
 - Cũng sẽ có 2 loại là by value và by reference.
 - Nếu là by value thì sẽ clone ra 1 con trỏ mới cũng sẽ trỏ vào cùng 1 vị trí với con trỏ cũ. Thì thấy rằng con trỏ có thể không thay đổi, nhưng mà giá trị được lưu ở địa chỉ đó có thể được thay đổi thông qua con trỏ clone
 - Pass by reference thì con trỏ có thể thay đổi theo.
- Pointer vs. array

```

//reference argument
void cal(int a[], int n);

//pointer argument
/**a with 2d array, ***a with 3d array,.....|
void cal(int *a, int n);

```

```

#include <iostream>
using namespace std;
int main() {
    int a[100];
    int *p = a;
    /*(p + i) is the a[i]
    /*(*(p + i) + j) is the a[i][j]
}

```

- Có 1 điểm đáng lưu ý ở đây là pass array vào trong function

```

void printArray(int a[ ], int n) {
    // cái mình pass vào trong này là 1 "con trỏ" lưu trữ cho vị trí đầu tiên
    // Bởi vì nên a++ tương đương với địa chỉ + 4 tương đương với phần tử
    // tiếp theo trong mảng
    for (int i = 0; i < n; i++)
        printf("%d ", *(a++)); // Same as a[i]...

    // nếu bây giờ bạn ghi sizeof(a) nó sẽ là 8 do a bây giờ là 1 con trỏ
}

int main() {
    int a[100];
    printArray(a, 100);
    for (int i = 0; i < n; i++)
        printf("%d ", *(a++)); // Wrong, why?
    // Ở đây a đang đóng vai trò là quản lý cả 1 mảng 100 phần tử vậy nếu
    // nó ++ lên thì nó địa chỉ → địa chỉ + 100
    // out khỏi mảng nên sai

    // nếu bạn ghi sizeof(a) ở đây thì là 4 * 100 = 400
}

```

- Dynamic memory

Hiểu về nó như thế nào đây???

Hẳn là bạn đã quen với cách khai báo `int a[100]`; mảng `a` chắc chắn có size cố định là 100 phần tử. Và nó sẽ được lưu trữ trong compile time.

Nhưng mà một số trường hợp chỉ muốn lưu 5 phần tử thì sao?? mấy cái còn lại sẽ không xài $\Rightarrow$ phí.

Vậy liệu có cách nào khởi tạo phụ thuộc vào ý chúng ta, hay nói chính xác hơn là trong quá trình chạy không??

Có 1 cách đó là sử dụng dynamic array thay vì sử dụng mảng tĩnh như mình hay dùng nữa.

ví dụ nếu mình muốn khai báo rằng: cấp cho tôi bộ nhớ để lưu trữ 5 phần tử hãy dùng

```
int *p = new int; //loại int, mấy loại khác cũng tương tự
```

```
int *a = new int[N]; //mảng N phần tử
```

Tuy nhiên rằng những dữ liệu này sẽ được lưu ở bộ nhớ heap và bạn hiểu đơn giản là nó sẽ không được xóa đi trừ khi bạn tắt máy $\Rightarrow$ nếu bạn không lấy lại những vị trí đó (xóa dữ liệu đi) thì sẽ gây ra vấn đề "memory leaking"

Cách để giải phóng:

```
delete p;  
delete[] a; //xóa với mảng
```

NOTE:Lưu ý rằng nếu có new thì bắt buộc delete, mọi hành vi quên đều dẫn đến hậu quả không lường trước được.

Sau khi đã biết được cái này rồi mình sẽ đưa ra cho bạn 1 đề bài: hãy tạo 1 struct lưu trữ 2 dữ liệu, đầu tiên là độ dài của xâu, thứ 2 là xâu.

Vậy để tránh lãng phí chúng ta sẽ sử dụng dynamic array:

```
struct nhan {  
    int n;  
    char *s; // bây giờ độ dài sẽ phụ thuộc vào số n bạn nhập  
}
```

```
n = 10;  
s = new char[n + 1]; //lưu ý có 1 vị trí cho null nhé!!!
```

Vấn đề lại xảy ra tiếp, bạn thấy rằng trong struct của mình đang lưu lại con trỏ đúng không???

Vậy nếu mình có:

```
nhan a_1, a_2;
```

a_1 đã có giá trị.

```
a_2 = a_1; đây là phép gán
```

vậy thì đúng rằng dữ liệu của a_1 đã được copy sang a_2, nhưng mà đó chỉ là con trỏ, cả 2 con trỏ đó đều trỏ đến cùng 1 mảng, từ đó dẫn đến a_2 mà thay đổi gì liên quan đến mảng $\Rightarrow$ a_2 thay đổi theo $\Rightarrow$ không hợp lý

Vậy nên nếu mà struct có con trỏ như thế này phải viết hàm gán hợp lý như sau:

```
void gan(const nhan a_1, nhan &a_2) { // đảm bảo a_1 ko thay đổi và a_2 có thay đổi  
    a_2.n = a_1.n;
```

```

a_2.s = new char[a_2.n + 1];
for(int i = 0; i < a_2.n; i++) {
    a_2.s[i] = a_1.s[i];
}
}

```

- Other types of pointers

- Constant pointer: chỉ trỏ đến 1 địa chỉ cho đến hết cuộc đời.

```

int x = 5, y = 6;
int * const p = &x;
p = &y; // Wrong.

```

- Pointer to constant: Memory content pointer points to cannot be changed.

```

int x = 5;
const int *p = &x;
*p = 6; // Wrong.

```

- void pointer: Pointer can store address of any types.

```

void printBytes(void *p, int size)
{
    char *q = ( unsigned char * ) p;
    for ( int i = 0; i < size; i++ )
        printf( "%d ", q[ i ] );
}

```

9. Linked list

- Hạn chế của array:
 - Vì array là 1 vùng dữ liệu liên tiếp, nên mình có thể random access (truy cập nhanh vào một phần tử).

- Tuy nhiên nếu bây giờ bạn cần tăng số lượng phần tử thì rất khó. Thêm với việc xóa đi 1 phần tử thì phải shift mảng. và việc set up 1 vùng dữ liệu liên tiếp lớn đôi khi là không khả thi. $\Rightarrow$ giải pháp là sử dụng linked list
- bây giờ chúng ta N vùng bất kì mà không cần liên tiếp, mỗi vùng đó sẽ lưu trữ dữ liệu của 1 đồ vật và địa chỉ có vùng tiếp theo $\Rightarrow$ mình chỉ cần biết địa chỉ của vùng đầu tiên thì sẽ biết hết danh sách của mình.
- Singly linked list:

```
struct node {
    //data
    node *nx;
};

node *head; // cần lưu trữ lại mỗi thằng đầu tiên là được.

phần tử cuối cùng có nx là nullptr
```

- Các thao tác ở trên linked list
 - Initialize: Ban đầu list của mình chưa có gì cả nên là head là nullptr
 - Check empty: chỉ cần xem head có bằng null hay không, nếu có thì list của mình rỗng
 - search: Để search thì mình cần duyệt qua mọi node nhưng mà do mình ko thể random access như là array nên là phải đi từ head, rồi từ head lấy dữ liệu đến thằng tiếp theo, cứ như vậy đến khi đến null thì dừng, hoặc khi tìm được thằng đó thì dừng.
 - add: Nếu mình thêm vào đầu thì đơn giản là đổi lại head bằng node thêm vào sau đó nx của node mới sẽ trỏ đến head cũ. Nếu như bây giờ thêm C vào giữa A và B thì bạn cần chỉnh lại nx của A là C và nx của C là B
 - remove: hiện tại là $A \rightarrow B \rightarrow C$, bạn cần xóa B thì đầu tiên cần tìm ra B, rồi lưu trữ lại node trước B và sau B, sau đó gán $A.nx = C$, rồi delete B
- Cái ở trên mình giới thiệu là single linked list, thì tại mỗi node thì chỉ cần lưu lại nx, còn 1 loại nữa là double linked list thì tại mỗi node sẽ lưu lại 2 cái là nx và

pre là node đứng trước nó và sau nó.

10. Stack and Queue

- Stack: Đây là dữ liệu như 1 cái hộp vậy, nếu bạn bỏ A và rồi bỏ B vào thì B sẽ được lấy ra trước thì mới lấy A ra được. LIFO(last in first out)
- Vậy liệu có thể sử dụng linked list để cài đặt stack không????
 - Vậy thì phải xem stack sẽ có những thao tác gì để mình build cho phù hợp: Có 2 thao tác chính là thêm vào đầu và lấy ra từ đầu. Vậy nên lúc build stack chúng ta sẽ làm như sau: với thao tác thêm vào thì mình sẽ thêm vào head của linked list, lấy ra cũng lấy head ra xong rồi đổi head của linked list thành thẳng nx của nó.

```
void push(int value) {  
  
    Node *tmp = new Node(value);  
    Node *cur = head;  
    head = tmp;  
    head → nx = cur;  
}  
  
void pop() {  
    //nhớ xét TH head là null  
  
    if(head == nullptr) return;  
  
    Node *tmp = head;  
    head = head → nx;  
    delete tmp;  
}
```

- Queue: Nó sẽ giống như một cái hộp chữ nhật 2 đầu, 1 đầu vào và 1 đầu ra thì nếu A vào trước B thì chắc chắn A phải ra trước B. FIFO(first in first out)

- Để cài đặt queue thì rắc rối hơn 1 xíu do phải quản lí cả 2 đầu nên chúng ta sẽ cần lưu lại cả head và tail(vùng lưu dữ liệu cuối cùng). Nếu mình add vào ở head và pop ra ở tail thì sẽ không hợp lí do sau khi mất đi tail cũ thì làm cách nào để tìm lại tail mới(lưu ý đang xài single linked list) ngoài cách chạy lại hết list??? Vậy thì mình sẽ chọn cách là thêm vào và pop ra ở head. Do mình có thể biết thẳng head mới chính là head cũ nx

```
void push(int value) {  
    Node *tmp = new node(value);  
    if(head == nullptr) {  
        head = tmp;  
        tail = tmp;  
        return;  
    }  
  
    tail->nx = tmp;  
    tail = tmp;  
}
```

```
void pop() {  
    if(head == null) return;  
    if(head->nx == null) {  
        delete head;  
        head = null;  
        tail = null;  
        return;  
    }  
    Node *tmp = head->nx;  
    delete head;  
    head = tmp;  
}
```